Digital Logic Circuit Design

VHDL Assignment-2
설계 및 검증 보고서

보고서 요약 강의자료(VHDL_1)의 VHDL 작성 방식과 Logisim VHDL Entity 구성 방식에 맞추어 5개 조합논리 문제를 설계하였다. 각 설계를 GHDL 테스트벤치, Logisim + QuestaSim 공동 시뮬레이션, Logisim GUI 직접 조작의 세 가지 방법으로 검증하였다.

과목	Digital Logic Circuit Design
이름	김우준
학번	2023092479
작성일	2026년 10월 26일

검증 요약: 5개 설계 모두 요구사항과 일치 (GHDL · Logisim/QuestaSim · GUI 확인)

1. 과제 개요

본 과제는 강의자료(VHDL_1)의 VHDL 기본 구조, multiplexer, 7-segment display, voting machine 예제를 바탕으로 5개의 조합논리 회로를 구현하는 것이다. 각 문제는 VHDL 소스(`.vhd`)와 Logisim 프로젝트(`.circ`)로 구성하였다.
Logisim 프로젝트는 강의자료와 같이 VHDL 코드를 **VHDL Entity 블록**으로 넣고, 입력 핀·DIP 스위치·스플리터·터널·7-segment·LED를 연결하는 방식으로 만들었다.

1.1 구현 항목

문제	설계 내용	Entity	핵심 출력 조건
1	4입력 AND / OR	logic_test	out1 = a·b·c·d, out2 = a+b+c+d
2	8-to-1 Multiplexer	mux8_1	ch(2:0)에 따라 a~h 중 하나를 data로 출력
3	7-segment 문자 표시	led_letter	dat에 따라 A, b, C, d, F, g 표시
4	25인 투표기	vote25	25표 중 13표 이상이면 result = 1
5	13-bit 1의 개수 표시	count_led	0~9는 숫자, 10~15는 A~F, en = 0이면 표시 꺼짐

1.2 개발 및 검증 환경

항목	환경
HDL	VHDL, `std_logic_1164` + `std_logic_arith` + `std_logic_unsigned` (강의자료와 동일)
회로 프로젝트	Logisim-evolution 5.0.0, VHDL Entity 블록
VHDL 시뮬레이터	Questa Altera Starter FPGA Edition 2025.2 (Logisim의 VHDL Simulation 기능과 연동)
테스트벤치 실행	GHDL 6.0.0 (mcode), `-fsynopsys`, `--assert-level=error`

1.3 검증 방법

#	방법	내용
①	GHDL 자가검증 테스트벤치	각 entity에 입력을 자동으로 넣고 기대값과 `assert`로 비교. 가능한 경우 모든 입력 조합을 전수 검사
②	Logisim + QuestaSim 공동 시뮬레이션	.circ 파일 안의 VHDL Entity를 Questa로 실행하고, Logisim 핀에 입력을 넣어 출력 확인 (총 514건)
③	Logisim GUI 직접 조작	VHDL Simulation을 켠 상태에서 Poke Tool로 핀·DIP 스위치를 직접 클릭하고 결과 화면을 캡처 (본문 그림)

1.4 강의자료 작성 방식 적용

강의자료 요소	강의 예제	본 과제 적용
library 3종, entity / `architecture bhv`, signal assignment <=	logic_test, mux2_1	5개 설계 공통 (문제 1은 <=만 사용)
`process` + `case ... when others`	mux4_1, led	문제 2, 3, 5
`if en='0'` → `"ZZZZZZZZ"` (Hi-Z)	led	문제 3, 5
`variable sum` + `for loop`	vote13	문제 4, 5
Logisim VHDL Entity + 입출력 장치 연결	logic_test, led, vote13	문제 1~5 회로

2. 문제 1 - 4입력 AND / OR

네 입력 a, b, c, d의 AND 결과를 out1에, OR 결과를 out2에 출력한다. 강의자료 1.2~1.3절의 `logic_test` 예제를 4입력으로 확장하였다.

입력 핀 a, b, c, d → **logic_test** → **출력 핀 out1, out2**

2.1 VHDL 코드 (logic_test.vhd)

```
--library and package
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

--entity
entity logic_test is
    port(
        a,b,c,d :in std_logic;
        out1,out2 :out std_logic
    );
end entity logic_test;

--architecture
architecture bhv of logic_test is
begin
    out1 <= a and b and c and d;
    out2 <= a or b or c or d;
end architecture bhv;
```

2.2 진리표 (대표값)

a b c d	out1 (AND)	out2 (OR)	설명
0000	0	0	모두 0
1000	0	1	하나만 1
0111	0	1	a만 0
1111	1	1	모두 1

2.3 Logisim 시뮬레이션 결과

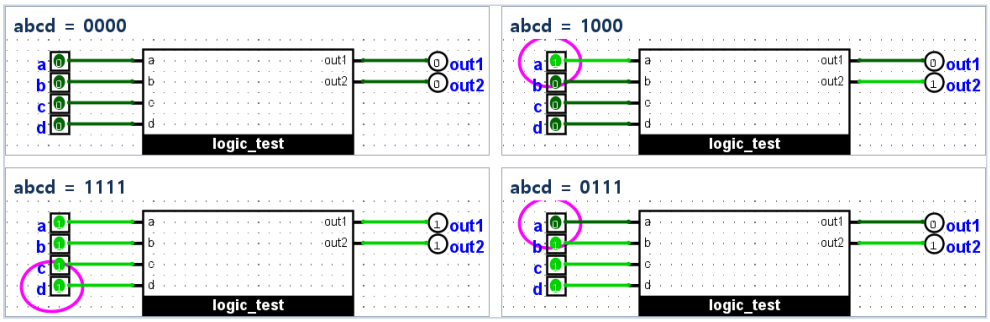


그림 1. Logisim에서 입력 핀을 직접 클릭한 결과 (밝은 초록 = 1, 진한 초록 = 0)

2.4 검증 결과

- GHDL: 16개 입력 조합 전수 검사, assertion error 0건 (160 ns)
- Logisim + Questa: 16개 입력 조합에서 out1, out2 모두 기대값과 일치
- GUI: 0000 / 1000 / 1111 / 0111에서 0/0, 0/1, 1/1, 0/1 확인 (그림 1)

PASS out1은 네 입력이 모두 1일 때만, out2는 하나라도 1이면 1이 된다.

3. 문제 2 - 8-to-1 Multiplexer

3-bit 선택 신호 ch가 000부터 111까지 바뀔 때 여덟 입력 a~h 중 하나를 data로 전달한다. 강의자료 2.2절 `mux4_1`의 `process + case` 구조를 8입력, 3-bit 선택으로 확장하였다.

입력 핀 a~h, ch(2:0) → **mux8_1** → **출력 핀 data**

3.1 선택표

ch	000	001	010	011	100	101	110	111
data	a	b	c	d	e	f	g	h

3.2 VHDL 코드 (mux8_1.vhd)

```
--library and package
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

--entity
entity mux8_1 is
    port(
        a,b,c,d,e,f,g,h :in std_logic;
        ch :in std_logic_vector(2 downto 0);
        data :out std_logic
    );
end entity mux8_1;

--architecture
architecture bhv of mux8_1 is
begin
    process(a,b,c,d,e,f,g,h,ch)
    begin
        case ch is
            when "000" => data <= a;
            when "001" => data <= b;
            when "010" => data <= c;
            when "011" => data <= d;
            when "100" => data <= e;
            when "101" => data <= f;
            when "110" => data <= g;
            when "111" => data <= h;
            when others => data <= '0';
        end case;
    end process;
end architecture bhv;
```

3.3 Logisim 시뮬레이션 결과

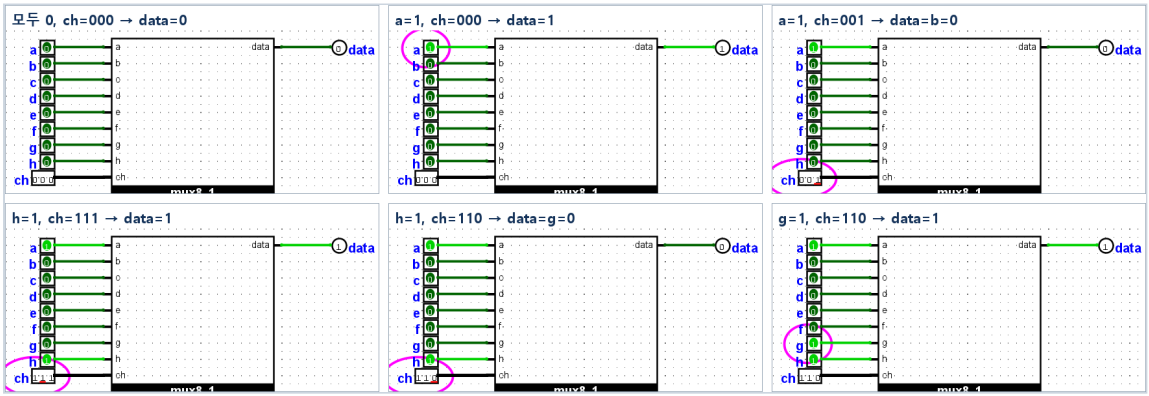


그림 2. ch 값을 바꾸며 선택된 입력이 data로 나오는지 확인한 결과

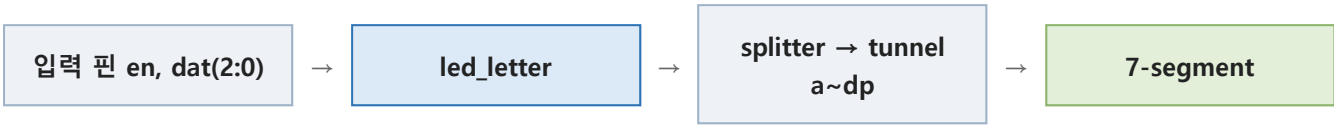
3.4 검증 결과

- GHDL: 데이터 256패턴 × 선택값 8개 = 2,048개 조합 전수 검사, assertion error 0건 (20,480 ns)
- Logisim + Questa: 무작위 데이터 20패턴 × 선택값 8개 = 160개 상태 일치
- GUI: ch = 000, 001, 111, 110에서 각각 a, b, h, g의 값이 data로 출력됨 (그림 2)

PASS 모든 선택값에서 data가 ch가 가리키는 입력과 일치한다.

4. 문제 3 - 7-segment 문자 표시

7-segment에 문자 a, b, c, d, f, g를 표시한다. 강의자료 3절의 `led` 예제와 같이 enable 입력 en과 선택 입력 dat를 두고, 출력 display(7:0)를 스플리터로 나누어 터널을 거쳐 7-segment에 연결하였다. 출력 비트 순서는 강의자료 진리표와 같은 **a b c d e f g dp**(display(7) = a, display(0) = dp)이며, 1이면 해당 segment가 켜진다.



4.1 문자 패턴

dat	표시	display (a b c d e f g dp)	켜지는 segment
000	A	11101110	a b c e f g
001	b	00111110	c d e f g
010	C	10011100	a d e f
011	d	01111010	b c d e g
100	F	10001110	a e f g
101	g	11110110	a b c d f g
110, 111	꺼짐	00000000	없음
en = 0	꺼짐	ZZZZZZZZ	없음 (Hi-Z)

a, c, f는 7-segment에서 흔히 쓰는 16진 표기(A b C d E F)와 같은 모양인 A, C, F로 표시하고, b, d, g는 소문자 모양으로 표시하였다.

4.2 VHDL 코드 (led_letter.vhd)

```
--library and package
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

--entity
--display = a b c d e f g dp
--dat : "000"=A, "001"=b, "010"=C, "011"=d, "100"=F,
"101"=g
entity led_letter is
    port(
        en :in std_logic;
        dat :in std_logic_vector(2 downto 0);
        display :out std_logic_vector(7 downto 0)
    );
end entity led_letter;

--architecture
architecture bhv of led_letter is
begin
    process(en,dat)
    begin
        if en='0' then
            display <= "ZZZZZZZZ";
        else
            case dat is
                when "000" => display <= "11101110";
--A
                when "001" => display <= "00111110";
--b
                when "010" => display <= "10011100";
--C
                when "011" => display <= "01111010";
--d
                when "100" => display <= "10001110";
--F
                when "101" => display <= "11110110";
--g
                when others => display <= "00000000";
            end case;
        end if;
    end process;
end architecture bhv;
```

4.3 Logisim 회로 및 시뮬레이션 결과

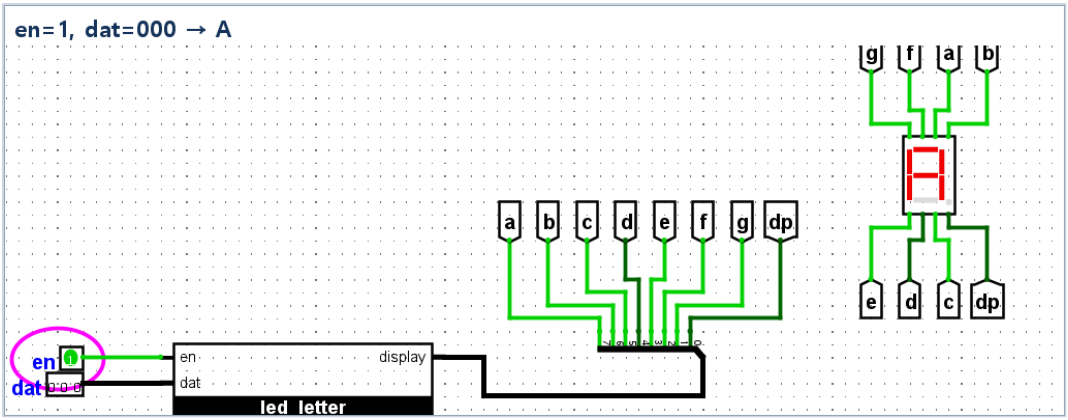


그림 3. 문제 3 Logisim 회로 (en=1, dat=000 → A). display 버스를 스플리터로 나누고 터널로 7-segment에 연결

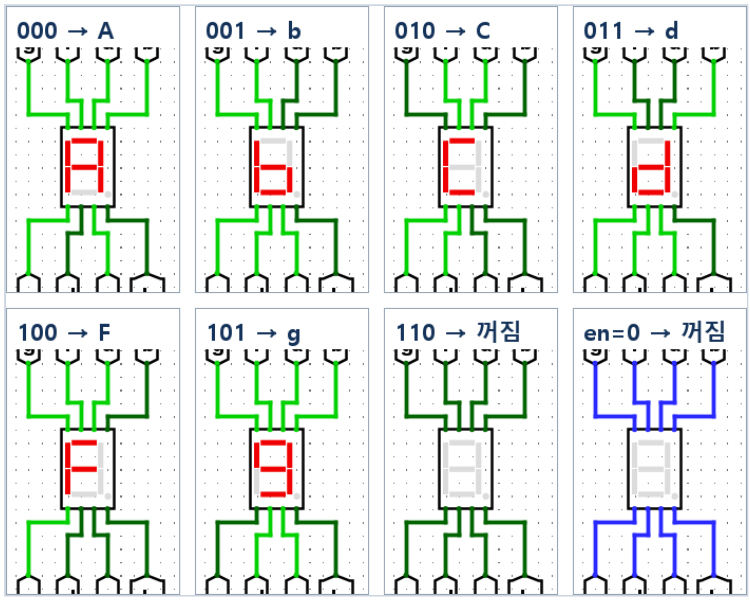


그림 4. dat 값에 따른 7-segment 표시. en=0에서는 출력이 Hi-Z(파란 선)가 되어 표시가 꺼진다

4.4 검증 결과

- GHDL: dat 8개 코드 × en 2가지 = 16개 상태 검사, assertion error 0건 (160 ns)
- Logisim + Questa: en=1의 8개 코드 모두 기대 패턴과 일치. en=0에서는 출력이 Logisim의 floating 값으로 나타나 Hi-Z 동작 확인
- GUI: A, b, C, d, F, g 표시와 110 및 en=0일 때 꺼짐 확인 (그림 4)

PASS 여섯 문자 패턴과 enable(Hi-Z) 동작이 모두 기대값과 일치한다.

5. 문제 4 - 25인 투표기

25명의 투표 입력 중 과반인 13표 이상이 1이면 result를 1로 출력한다. 강의자료 4절의 `vote13` 예제와 같이 `variable sum`과 `for` 반복문으로 1의 개수를 센다. Logisim에서는 25-way DIP 스위치를 스플리터로 묶어 vt(25 downto 1)에 연결하고(스위치 k번 → vt(k)), 결과를 LED로 표시하였다.



5.1 VHDL 코드 (vote25.vhd)

```
--library and package
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

--entity
entity vote25 is
    port(
        vt :in std_logic_vector(25 downto 1);
        result :out std_logic
    );
end entity vote25;

--architecture
architecture bhv of vote25 is
begin
    process(vt)
    variable sum:integer range 0 to 25;
    begin
        sum:= 0;
        for i in 1 to 25 loop
            if (vt(i)='1') then
                sum:=sum+1;
            end if;
        end loop;
        if sum >= 13 then
            result <= '1';
        else
            result <= '0';
        end if;
    end process;
end architecture bhv;
```

5.2 경계 조건

찬성표 수	0 ~ 12	13	14 ~ 25
result	0	1	1

5.3 Logisim 회로 및 시뮬레이션 결과



그림 5. 경계값 확인. 12표에서는 LED가 꺼지고, 13표에서 LED가 켜진다

5.4 검증 결과

- GHDL: 0~25표를 하위·상위 비트에 각각 배치한 52개 경계 패턴 + 32-bit LFSR로 만든 무작위 2,000개 패턴, assertion error 0건 (20,520 ns)
- Logisim + Questa: 경계 패턴 52개 + 무작위 150개 = 202개 상태 일치
- GUI: 11표·12표에서 LED 꺼짐, 13표·14표에서 LED 켜짐 확인 (그림 5)

PASS 12표에서 0, 13표 이상에서 1이 출력되어 과반 조건을 정확히 만족한다.

6. 문제 5 - 13-bit 입력의 1 개수 표시

13-bit 입력 vt에 있는 '1'의 개수를 세어 7-segment에 표시한다. 개수가 0~9이면 숫자, 10~15이면 A, b, C, d, E, F를 표시하고, en = 0이면 표시를 끈다. 문제 4의 개수 세기(`vote13` 방식)와 문제 3의 7-segment 출력(`led` 방식)을 결합하였다.



6.1 VHDL 코드 (count_led.vhd)

```
--library and package
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

--entity
--display = a b c d e f g dp
entity count_led is
    port(
        en :in std_logic;
        vt :in std_logic_vector(13 downto 1);
        display :out std_logic_vector(7 downto 0)
    );
end entity count_led;

--architecture
architecture bhv of count_led is
begin
    process(en,vt)
    variable sum:integer range 0 to 15;
    begin
        sum:= 0;
        for i in 1 to 13 loop
            if (vt(i)='1') then
                sum:=sum+1;
            end if;
        end loop;
        if en='0' then
            display <= "ZZZZZZZZ";
        else
            case sum is
                when 0 => display <= "11111100";
                when 1 => display <= "01100000";
                when 2 => display <= "11011010";
                when 3 => display <= "11110010";
                when 4 => display <= "01100110";
                when 5 => display <= "10110110";
                when 6 => display <= "10111110";
                when 7 => display <= "11100000";
                when 8 => display <= "11111110";
                when 9 => display <= "11100110";
                when 10 => display <= "11101110"; --A
                when 11 => display <= "00111110"; --b
                when 12 => display <= "10011100"; --C
                when 13 => display <= "01111010"; --d
                when 14 => display <= "10011110"; --E
                when 15 => display <= "10001110"; --F
                when others => display <= "00000000";
            end case;
        end if;
    end process;
end architecture bhv;
```

6.2 출력 패턴

개수	0	1	2	3	4	5	6	7
표시	0	1	2	3	4	5	6	7
display (hex)	FC	60	DA	F2	66	B6	BE	E0
개수	**8**	**9**	**10**	**11**	**12**	**13**	**14**	**15**
표시	8	9	A	b	C	d	E	F
display (hex)	FE	E6	EE	3E	9C	7A	9E	8E

display 값은 a b c d e f g dp 순서의 8-bit 값을 16진수로 나타낸 것이다. vt가 13-bit이므로 실제 개수의 최댓값은 13(d)이며, E(14)와 F(15)는 과제 요구에 맞춰 case 항목으로만 구현하였다.

6.3 Logisim 회로 및 시뮬레이션 결과

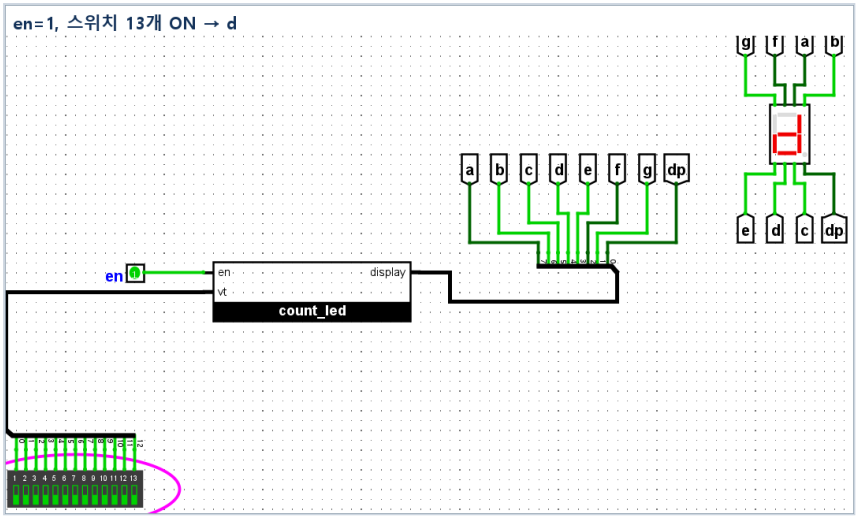


그림 6. 문제 5 Logisim 회로 (en=1, DIP 스위치 13개 모두 ON → d)

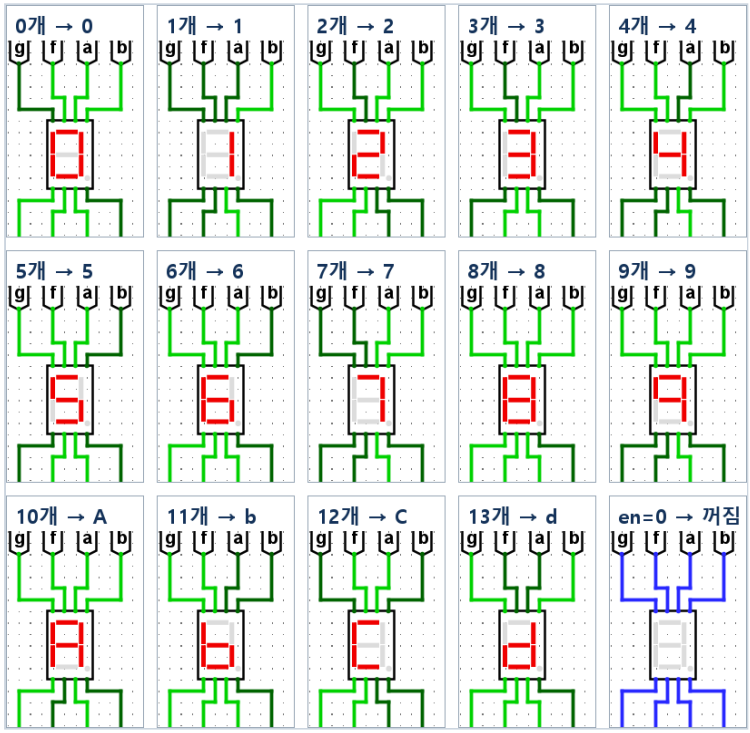


그림 7. 켠 스위치 개수에 따른 표시 (0~9, A, b, C, d)와 en=0일 때 꺼짐

6.4 검증 결과

- GHDL: vt의 8,192개 패턴 전체를 en=0, en=1에서 각각 검사(16,384개 상태), assertion error 0건 (163,840 ns)
- Logisim + Questa: 개수 0~13마다 서로 다른 스위치 조합 6개씩(84개) + en=0 20개 = 104개 상태 확인
- GUI: 스위치를 하나씩 켜며 0~9, A, b, C, d가 차례로 표시되고, en=0에서 꺼지는 것 확인 (그림 7)

PASS 가능한 모든 13-bit 입력에서 1의 개수와 7-segment 표시가 일치한다.

7. 종합 검증 결과

문제	① GHDL 테스트벤치	② Logisim + Questa	③ GUI 직접 조작	결과
P1 logic_test	16 조합 (전수)	16 조합 × 출력 2	4 상태	PASS
P2 mux8_1	2,048 조합 (전수)	160 상태	6 상태	PASS
P3 led_letter	16 상태 (전수)	16 상태	8 상태	PASS
P4 vote25	2,052 패턴	202 상태	4 상태	PASS
P5 count_led	16,384 상태 (전수)	104 상태	15 상태	PASS

PASS 5개 설계 모두 컴파일·시뮬레이션에 성공했고, 세 가지 검증 방법 모두에서 오류가 없었다.

- 각 .circ 파일 안의 VHDL Entity 코드는 제출용 .vhd 파일과 동일하다.
- DIP 스위치 k번이 vt(k)로, display(7)이 segment a로 연결되는 것을 Logisim에서 확인하였다.

8. 설계 시 고려사항

항목	내용
숫자 5의 segment 패턴	강의자료 7-segment 진리표의 5는 `10110110`이지만, 같은 자료의 `led` 예제 코드에는 6과 같은 `10111110`으로 적혀 있다. 진리표에 맞는 `10110110`을 사용하였다.
투표기 비교문 위치	강의자료 `vote13` 코드는 `if vt(i)='1'` 안에서만 result를 정하므로, 모든 입력이 0이면 result가 갱신되지 않는다. 같은 문법을 쓰되 개수 비교를 반복문이 끝난 뒤로 옮겨 모든 입력에서 result가 정해지도록 하였다.
표시 끄기(en = 0)	강의자료 `led` 예제와 같이 `"ZZZZZZZZ"`(Hi-Z)로 출력을 끊어 7-segment를 끈다. 문제 3도 같은 `led` 구조를 따라 en 입력을 두었다.
문제 5의 개수 판별	개수(정수 sum)에 따라 패턴을 고르기 위해 `case sum is when 0 => ...`처럼 정수 값에 대한 case를 사용하였다(강의자료의 case 문을 정수에 적용).

9. 결론

다섯 문제 모두 강의자료에서 다룬 entity / architecture 구조, signal assignment, process, case, if, for 반복문과 7-segment 표현 방식으로 구현하였다. Logisim 프로젝트도 강의자료와 같이 VHDL Entity 블록에 입력 핀, DIP 스위치, 스플리터, 터널, 7-segment, LED를 연결하는 형태로 구성하였다.

GHDL 테스트벤치에서는 문제 1, 2, 3, 5의 모든 입력 조합과 문제 4의 경계·무작위 패턴을 오류 없이 통과하였다. Logisim과 QuestaSim 공동 시뮬레이션, Logisim 화면에서의 직접 조작에서도 모든 결과가 요구사항과 일치하였다. 특히 문제 4는 12표와 13표의 경계를 정확히 구분하고, 문제 5는 가능한 모든 13-bit 입력에서 올바른 숫자·문자를 표시한다. 따라서 본 설계는 과제 명세의 기능 요구사항을 모두 충족한다.

9.1 제출 파일 구성

폴더	VHDL 소스	Logisim 프로젝트
P1_logic_test	logic_test.vhd	logic_test.circ
P2_mux8_1	mux8_1.vhd	mux8_1.circ
P3_led_letter	led_letter.vhd	led_letter.circ
P4_vote25	vote25.vhd	vote25.circ
P5_count_led	count_led.vhd	count_led.circ
testbench	tb_*.vhd 5개 (검증용)	-
(루트)	VHDL_Assignment_2_Report.pdf (본 보고서)	

9.2 Logisim에서 실행하는 방법

1. .circ 파일을 Logisim-evolution에서 연다.
2. 메뉴 **Simulate → VHDL Simulation Enabled**를 선택한다 (QuestaSim이 실행된다).
3. **Ctrl + R**(Reset Simulation)을 눌러 회로 값을 초기화한다.
4. Poke Tool(손 모양)로 입력 핀이나 DIP 스위치를 클릭하여 입력을 바꾼다.

9.3 참고자료

- VHDL_1.pdf - Digital Logic Circuit Design 강의자료 (Biomimetic Robotics Lab)
- Logisim-evolution 5.0.0, Questa Altera Starter FPGA Edition 2025.2, GHDL 6.0.0